

Advanced Graph RAG Assignment (2-Day Build)

Tech Stack: TypeScript, Next.js, NestJS, Neo4j (Graph DB), Vector DB (pgvector/Pinecone), Redis (optional), LLM API.

Objective

Build a production-style Graph-based Retrieval-Augmented Generation system that combines knowledge graphs and semantic retrieval to answer user queries with structured reasoning and citations.

Core Features

1. Document ingestion (PDF, TXT, Markdown).
2. Entity and relationship extraction from documents.
3. Knowledge graph construction (nodes: entities, edges: relationships).
4. Embedding generation for nodes and/or text chunks.
5. Hybrid retrieval: graph traversal + vector similarity.
6. Answer generation using LLM with structured + unstructured context.
7. Chat interface with response streaming and source attribution.

Graph RAG Pipeline

User Query → Entity Extraction → Graph Traversal → Subgraph Retrieval → Vector Search → Context Assembly → LLM → Answer + Sources.

Day 1 Deliverables

- Backend setup using NestJS.
- File upload and parsing pipeline.
- Entity extraction (NER via LLM or library).
- Graph construction in Neo4j.
- Embedding generation and storage.
- Ability to query vector DB independently.

Day 2 Deliverables

- Graph traversal API (based on entities in query).
- Hybrid retrieval combining graph + semantic search.
- Context builder for LLM prompts.
- Chat API with streaming responses.
- Frontend (Next.js) chat interface.
- Display retrieved sources and graph paths.

Prompt Design

Ensure the LLM is grounded by strictly using retrieved context. Include both graph-derived facts and semantic chunks.

Advanced Enhancements (Optional)

- Re-ranking retrieved nodes using LLM or cross-encoder.
- Multi-hop graph traversal.
- Query expansion using LLM.
- Caching embeddings and responses (Redis).
- Visualization of graph paths in UI.

Evaluation Criteria

Must Have: End-to-end Graph RAG pipeline, working UI, correct retrieval.

Good: Source attribution, clean architecture, modular services.

Advanced: Hybrid ranking, performance optimization, explainability.

Expected Output

User asks a question → system returns grounded answer along with:

- Supporting documents
- Graph relationships used
- Relevant extracted entities

Submission

Provide a GitHub repository with README including setup instructions, architecture explanation, and design decisions.